

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

«Вятский государственный университет»
(ФГБОУ ВО «ВятГУ»)

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт

Лабораторная работа №5 по дисциплине

«Методы обработки и анализа медицинских изображений»

на тему: «Классификация изображений с применением сверточных нейронных
сетей»

Выполнил студент группы ИВТм-1301-01-00 Лоленко Никита Вячеславович _____/

Проверил преподаватель Медведева Е. В. _____/

1. Цель работы

Цель работы: изучить методы построения и обучения сверточных нейронных сетей для классификации изображений.

2. Задание

Разработать прототип интеллектуальной системы с применением сверточной нейронной сети для классификации изображений.

Для варианта 2 был выбран набор данных Fashion-MNIST. Набор содержит изображения предметов одежды размером 28 на 28 пикселей. Всего используется 10 классов: футболка, брюки, пуловер, платье, пальто, сандалия, рубашка, кроссовок, сумка и ботинок.

3. Ход работы

3.1. Загрузка и разбиение набора данных

В работе использовался встроенный набор данных Fashion-MNIST из библиотеки Keras. Набор данных уже содержит готовые обучающую и тестовую части, поэтому отдельно выполнять ручное разбиение не требовалось.

Данные были нормализованы: значения пикселей приведены к диапазону от 0 до 1. Для этого исходные значения яркости от 0 до 255 были разделены на 255.

3.2. Аугментация

По рекомендации к работе исходная обучающая выборка была дополнена тремя дубликатами. Для дубликатов применялись простые преобразования:

- зеркальное отражение;
- сдвиг изображения вниз на 5 пикселей;
- осветление изображения по формуле $x * 0.68 + 0.17$.

После аугментации обучающая выборка стала в 4 раза больше: к исходным изображениям были добавлены три измененные копии. Ответы для обучающих данных также были повторены 4 раза.

Пример полученных копий показан на рисунке 1.

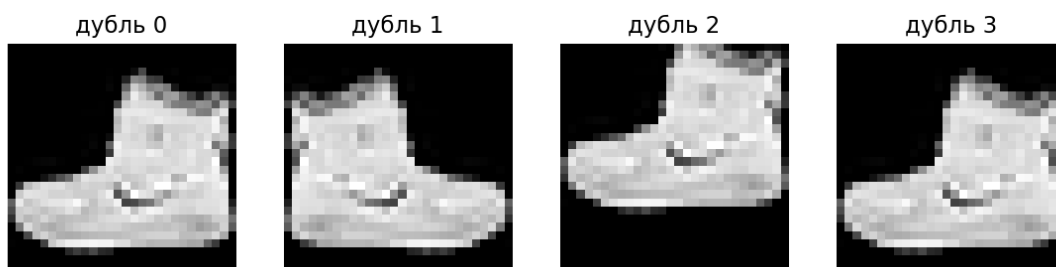


Рис. 1: Пример аугментации изображений

3.3. Архитектура модели

Была создана простая сверточная нейронная сеть на TensorFlow/Keras. В архитектуре использованы слои Conv2D, MaxPooling2D, Dropout, Flatten и Dense.

Архитектура модели:

- входной слой для изображения размером 28 на 28 пикселей с одним каналом;
- сверточный слой Conv2D с 17 фильтрами и ядром 5 на 5;
- слой MaxPooling2D;
- сверточный слой Conv2D с 31 фильтром и ядром 3 на 3;
- слой MaxPooling2D;
- слой Dropout со значением 0.27;
- слой Flatten;
- полносвязный слой Dense с 83 нейронами;
- слой Dropout со значением 0.43;
- выходной слой Dense на 10 классов с функцией активации softmax.

3.4. Обучение

Модель обучалась 11 эпох с размером батча 112. В качестве функции потерь использовалась sparse categorical crossentropy, оптимизатор: Adam со скоростью обучения 0.00063. После обучения веса модели были сохранены в файл `variant2_fashion.weights.h5`. Также веса дополнительно сохранялись в текстовый файл `weights.txt`, чтобы их можно было просмотреть.

Графики обучения приведены на рисунке 2.

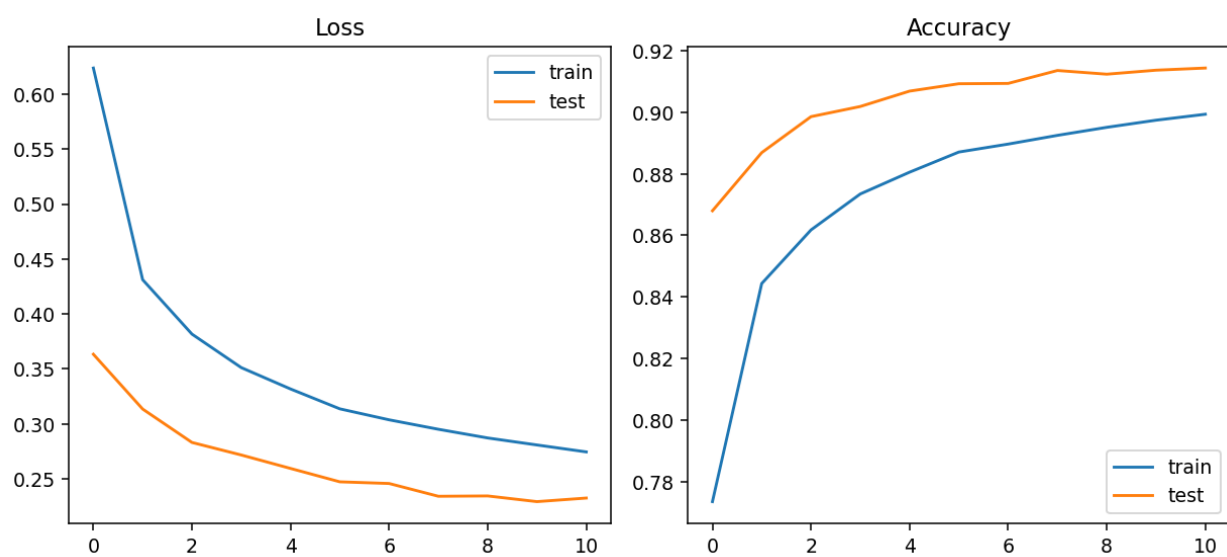


Рис. 2: Графики функции потерь и точности

4. Результаты

После обучения были получены следующие значения метрик на тестовой выборке:

Метрика	Значение
Accuracy	0.9144
Precision	0.9139
Recall	0.9144

Матрица ошибок показана на рисунке 3.

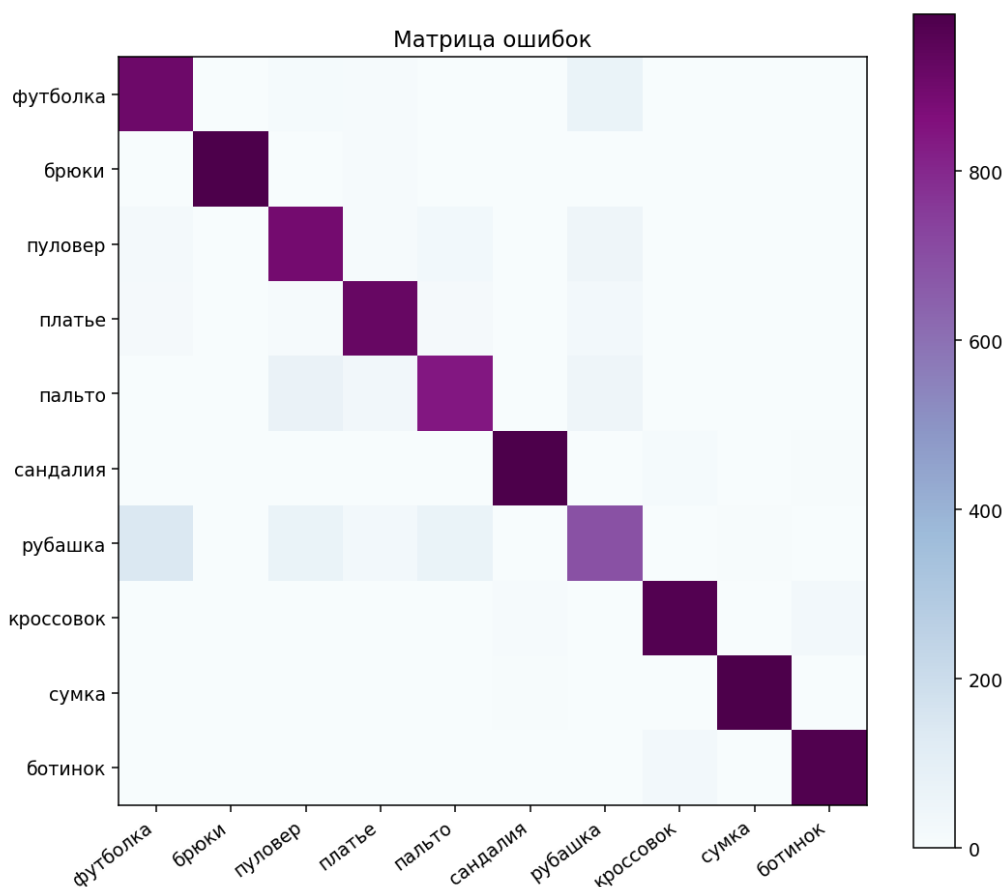


Рис. 3: Матрица ошибок

Примеры работы модели на изображениях из тестовой выборки приведены на рисунке 4.

Ошибки модели в основном связаны с визуально похожими классами одежды. Например, модель может путать футболку, рубашку, пуловер и пальто, так как эти изображения имеют похожую форму.

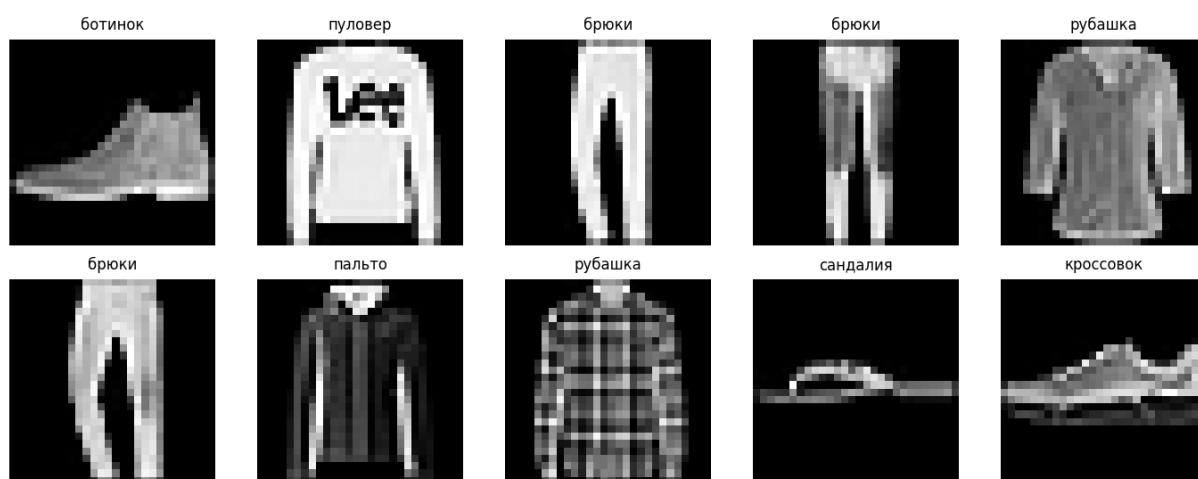


Рис. 4: Примеры предсказаний модели

5. Ответы на контрольные вопросы

1. Нейрон в искусственной сети получает входные значения, умножает их на веса, добавляет смещение и передает результат через функцию активации.
2. Классификация изображений - это задача определения класса объекта по входному изображению.
3. Transfer learning - это использование заранее обученной модели как основы для новой задачи.
4. Заморозка весов нужна, чтобы сохранить уже найденные признаки и обучать только новые слои.
5. GlobalAveragePooling2D усредняет карту признаков и превращает ее в компактный вектор.
6. Dropout случайно отключает часть нейронов во время обучения и снижает переобучение.
7. ReLU обнуляет отрицательные значения, sigmoid дает диапазон от 0 до 1, tanh дает диапазон от -1 до 1.
8. Градиентный спуск - это способ менять веса модели так, чтобы уменьшать функцию потерь.
9. Переобучение возникает, когда модель хорошо запоминает обучающие данные, но хуже работает на новых. Для борьбы используют аугментацию, Dropout и проверочную выборку.
10. Рекуррентные нейронные сети работают с последовательностями и учитывают предыдущее состояние.
11. CNN использует свертки и ищет локальные признаки, а полносвязная сеть соединяет все входы со всеми нейронами.
12. Сверточный слой выделяет признаки изображения: края, контуры, текстуры и их сочетания.
13. MaxPooling уменьшает размер карты признаков и оставляет самые сильные значения.

14. Эпоха - один проход по всей обучающей выборке, шаг - обработка одного батча.
15. Функция потерь показывает ошибку модели, а оптимизатор меняет веса для ее уменьшения.

6. Вывод

В ходе лабораторной работы была реализована сверточная нейронная сеть для классификации изображений Fashion-MNIST. Были выполнены загрузка готовых обучающей и тестовой выборок, нормализация, аугментация обучающей выборки тремя дополнительными копиями, обучение модели, сохранение весов, построение графиков и оценка качества по метрикам accuracy, precision и recall.

Полученная точность на тестовой выборке составила 0.9144. Это показывает, что простая сверточная сеть успешно решает задачу классификации предметов одежды из набора Fashion-MNIST.

7. Листинг программы

Ниже приведен основной файл программы.

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf

results = Path("results")
models = Path("models")

names = [
    "футболка", "брюки", "пуловер", "платье", "пальто",
    "сандалия", "рубашка", "кроссовок", "сумка", "ботинок",
]

# вариант 2 - Fashion-MNIST
(train_x, train_y), (test_x, test_y) = tf.keras.datasets.fashion_mnist.load_data() #
↳ train_x - для обучения, test_x - для проверки, train_y - правильные ответы
train_x = train_x.astype("float32") / 255
test_x = test_x.astype("float32") / 255

# три дубликата с простыми преобразованиями
mirror = np.flip(train_x, axis=2) # зеркальные картинки
drop_down = np.pad(train_x[:, 5:, :], ((0, 0), (0, 5), (0, 0))) # сдвинуты вниз
pale = np.clip(train_x * 0.68 + 0.17, 0, 1) # осветление

train_x = np.concatenate([train_x, mirror, drop_down, pale])[..., None] # объединяем все
train_y = np.concatenate([train_y, train_y, train_y, train_y]) # ответы увеличить в 4
↳ раза, так как 4 раза больше картинок
test_x = test_x[..., None] # черно белые

plt.figure(figsize=(8.3, 2.1))
for i, img in enumerate([train_x[0], mirror[0], drop_down[0], pale[0]]):
    plt.subplot(1, 4, i + 1)
    plt.imshow(img.squeeze(), cmap="gray")
    plt.title(f"дубль {i}")
    plt.axis("off")
plt.tight_layout()
plt.savefig(results / "augmentation.png", dpi=137)
plt.close()

# создание нейронки - слои поочередно
net = tf.keras.Sequential([
    tf.keras.layers.Input((28, 28, 1)), # входной слой, 28 на 28 пикселей, 1 канал
    ↳ (черно-белые)
```

```

tf.keras.layers.Conv2D(17, 5, padding="same", activation="relu"), # сверточный с 17
    ↪ фильтрами, ищет формы
tf.keras.layers.MaxPooling2D(), # сжимает мелкие детали
tf.keras.layers.Conv2D(31, 3, padding="same", activation="relu"),
tf.keras.layers.MaxPooling2D(),
tf.keras.layers.Dropout(0.27), # отключает 27% нейронов, чтобы не переобучаться
tf.keras.layers.Flatten(), # данные из картинки в вектор
tf.keras.layers.Dense(83, activation="relu"), # принимает решение на основе найденных
    ↪ признаков, 83 нейрона
tf.keras.layers.Dropout(0.43),
tf.keras.layers.Dense(10, activation="softmax"), # 10 нейронов для 10 классов одежды,
    ↪ softmax превращает в вероятности
])

# настройка перед обучением
net.compile(
    optimizer=tf.keras.optimizers.Adam(0.00063), # меняет веса во время обучения, 0.00063
    ↪ - скорость обучения. если много - перепрыгнет решение, если мало - долго будет
    ↪ учиться
    loss="sparse_categorical_crossentropy", # насколько модель ошиблась(лосс такой, чтобы
    ↪ выбрать один класс из нескольких)
    metrics=["accuracy"], # показывать точность на обучении
)

# запуск обучения
story = net.fit(
    train_x,
    train_y,
    epochs=11, # сколько раз пройти по обучающим данным
    batch_size=112, # 112 картинок за раз
    validation_data=(test_x, test_y), # проверять точность на тестовых данных после
    ↪ каждой эпохи
)

net.save_weights(str(models / "variant2_fashion.weights.h5"))

with (models / "weights.txt").open("w", encoding="utf-8") as file:
    for layer in net.layers:
        weights = layer.get_weights()
        if not weights:
            continue

        file.write(f"Слой: {layer.name}\n")
        for number, values in enumerate(weights, start=1):
            file.write(f"Массив {number}, размер: {values.shape}\n")
            file.write(np.array2string(values, precision=6, suppress_small=True,
            ↪ threshold=np.inf))
            file.write("\n\n")

plt.figure(figsize=(9.1, 4.2))

```

```

plt.subplot(1, 2, 1)
plt.plot(story.history["loss"], label="train")
plt.plot(story.history["val_loss"], label="test")
plt.title("Loss")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(story.history["accuracy"], label="train")
plt.plot(story.history["val_accuracy"], label="test")
plt.title("Accuracy")
plt.legend()
plt.tight_layout()
plt.savefig(results / "graphs.png", dpi=137)
plt.close()

# предсказание на тестовых картинках
answer = net.predict(test_x).argmax(axis=1) # выбирает номер класса фотки с самой большой
↳ вероятностью
table = tf.math.confusion_matrix(test_y, answer, num_classes=10).numpy() # матрица ошибок

precision = np.diag(table) / (table.sum(axis=0) + 0.000013) # насколько можно доверять
↳ ответу
recall = np.diag(table) / (table.sum(axis=1) + 0.000013) # насколько хорошо модель
↳ находит реальные объекты классов
accuracy = (answer == test_y).mean() # общая точность

print("accuracy =", round(accuracy, 5))
print("precision =", round(precision.mean(), 5))
print("recall =", round(recall.mean(), 5))

(results / "metrics.txt").write_text(
    f"accuracy = {accuracy:.5f}\n"
    f"precision = {precision.mean():.5f}\n"
    f"recall = {recall.mean():.5f}\n",
    encoding="utf-8",
)

plt.figure(figsize=(8.2, 7.1))
plt.imshow(table, cmap="BuPu")
plt.title("Матрица ошибок")
plt.xticks(range(10), names, rotation=37, ha="right")
plt.yticks(range(10), names)
plt.colorbar()
plt.tight_layout()
plt.savefig(results / "matrix.png", dpi=137)
plt.close()

plt.figure(figsize=(10.3, 4.1))
for i in range(10):
    plt.subplot(2, 5, i + 1)

```

```
plt.imshow(test_x[i].squeeze(), cmap="gray")
plt.title(names[answer[i]], fontsize=8.7)
plt.axis("off")
plt.tight_layout()
plt.savefig(results / "check.png", dpi=137)
plt.show()
```